



아마존 리뷰를 이용한 상품 추천 시스템

KUGGLE 2차 프로젝트 - 문형주 강지윤 이서경 이현정

0

목차

1

서론

2

데이터
분석

3

데이터
전처리

4

모델링

5

하이퍼
파라미터
튜닝

6

결론

Chapter 1

서론

1

프로젝트의 배경 및 목표

프로젝트 배경 및 목표

배경

폭발적인 온라인 거래 시장의
성장과 함께, 사용자가 원하는
상품을 쉽게 찾을 수 있도록
돕는 개인화된 추천의 중요성 증대

목표

사용자의 미래 행동을 예측하여
개인화된 상품을 제공함으로써,
서비스 만족도 및 매출 증대에
기여하는 추천 시스템을 개발

Chapter 2

데이터 분석

1

데이터 탐색(EDA)

데이터 로드, 상위 5개 미리보기

```
df = pd.read_csv('/content/ratings_Electronics.csv', names=['userId',  
                                                         'productId', 'rating', 'timestamp'], nrows=None)  
  
print("데이터 원본 (상위 5개):")  
print(df.head())
```

데이터 원본 (상위 5개):

	userId	productId	rating	timestamp
0	AKM1MP6P00YPR	0132793040	5.0	1365811200
1	A2CX7LU0HB2NDG	0321732944	5.0	1341100800
2	A2NWSAGRHC8N5	0439886341	1.0	1367193600
3	A2WNB0D3WWDNKT	0439886341	3.0	1374451200
4	A1GI0U4ZRJA8WN	0439886341	1.0	1334707200

결측치 확인

```
df.isnull().sum()
```

	0
userId	0
productId	0
rating	0
timestamp	0

dtype: int64

→ 모든 컬럼에 결측치 X

데이터 탐색(EDA)

기본 정보

```
df.info()
```

— 데이터 정보 —

```
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 7824482 entries, 0 to 7824481
```

```
Data columns (total 4 columns):
```

#	Column	Dtype
---	--------	-------

0	userId	object
---	--------	--------

1	productId	object
---	-----------	--------

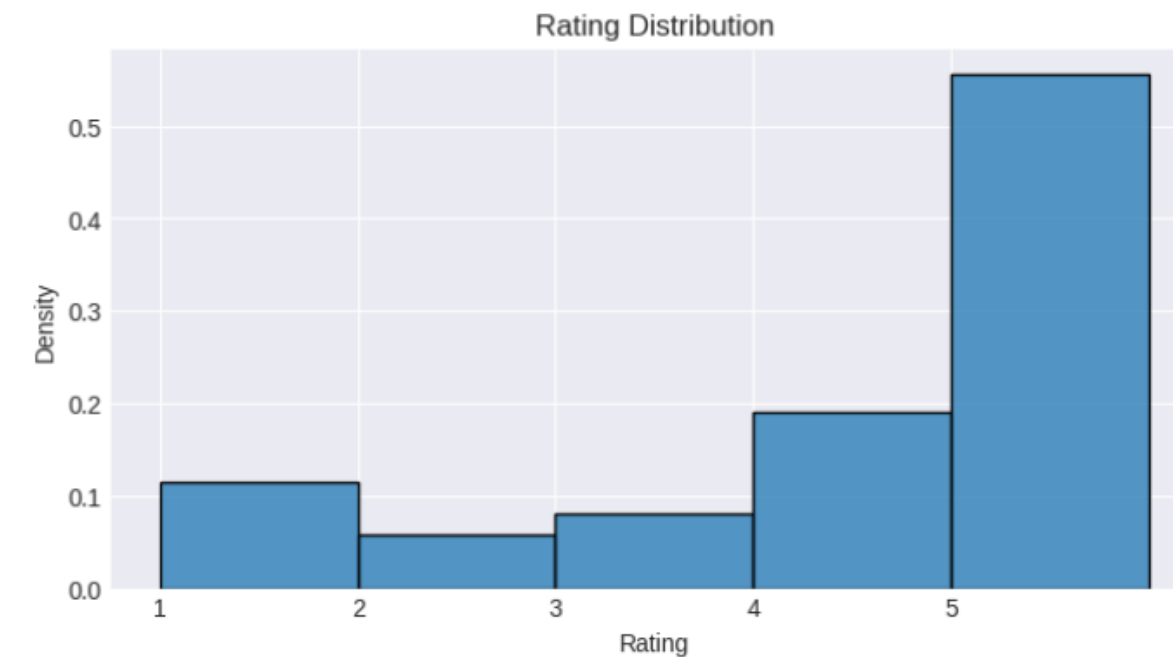
2	rating	float64
---	--------	---------

3	timestamp	int64
---	-----------	-------

```
dtypes: float64(1), int64(1), object(2)
```

```
memory usage: 238.8+ MB
```

평점 분포



데이터 희소성(Sparsity)

사용자 수: 4201696, 아이템 수: 476002

총 상호작용 수: 7824482

데이터 희소성 (Sparsity): 0.999996

→ 0.0004%만 실제로 채워져 있음

- 대부분의 사용자는 리뷰를 거의 남기지 않음
- 대부분의 상품은 일부 사용자만 평가

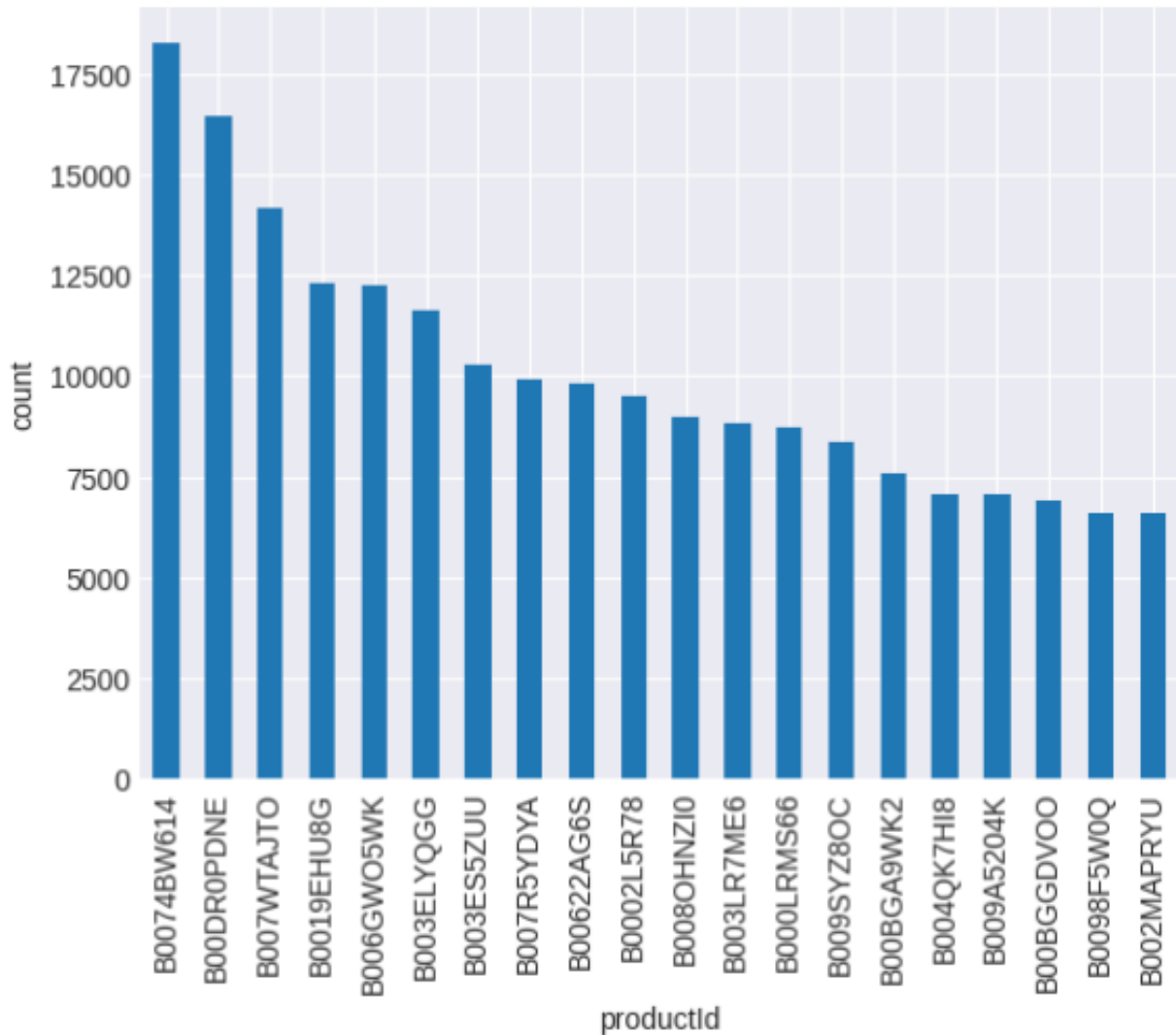
상위 인기 아이템 & 적극적인 사용자

Top 20 인기 상품:

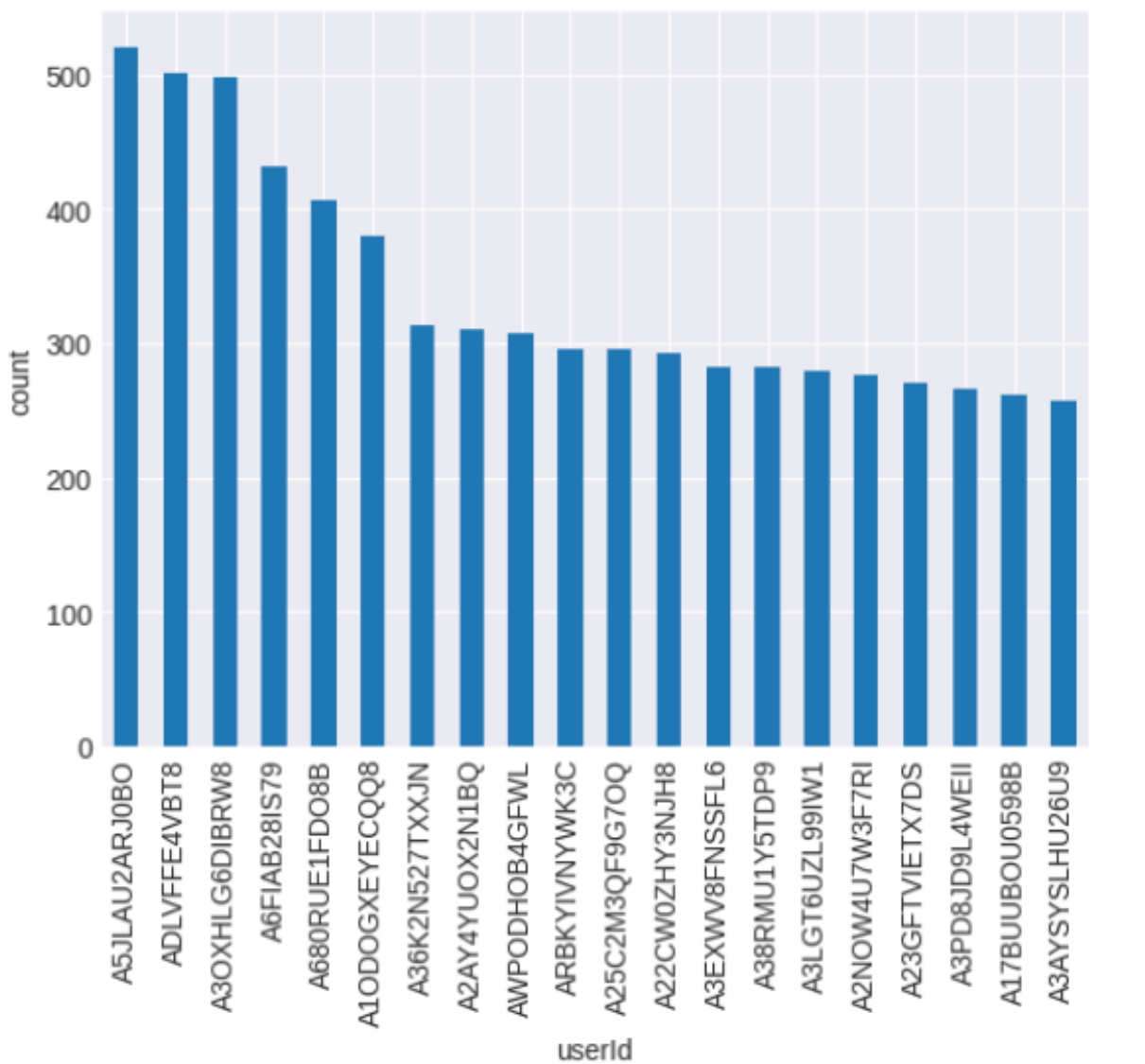
productId	count
B0074BW614	18244
B00DR0PDNE	16454
B007WTAJT0	14172
B0019EHU8G	12285
B006GW05WK	12226
B003ELYQGG	11617
B003ES5ZUU	10276
B007R5YDYA	9907
B00622AG6S	9823
B0002L5R78	9487
B0080HNZIO	8966
B003LR7ME6	8840
B000LRMS66	8715
B009SYZ8OC	8370
B00BGA9WK2	7561
B004QK7HI8	7060
B009A5204K	7059
B00BGGDV00	6893
B0098F5W0Q	6616
B002MAPRYU	6599

Name: count, dtype: int64

〈상품별 평가 수 상위 20개〉

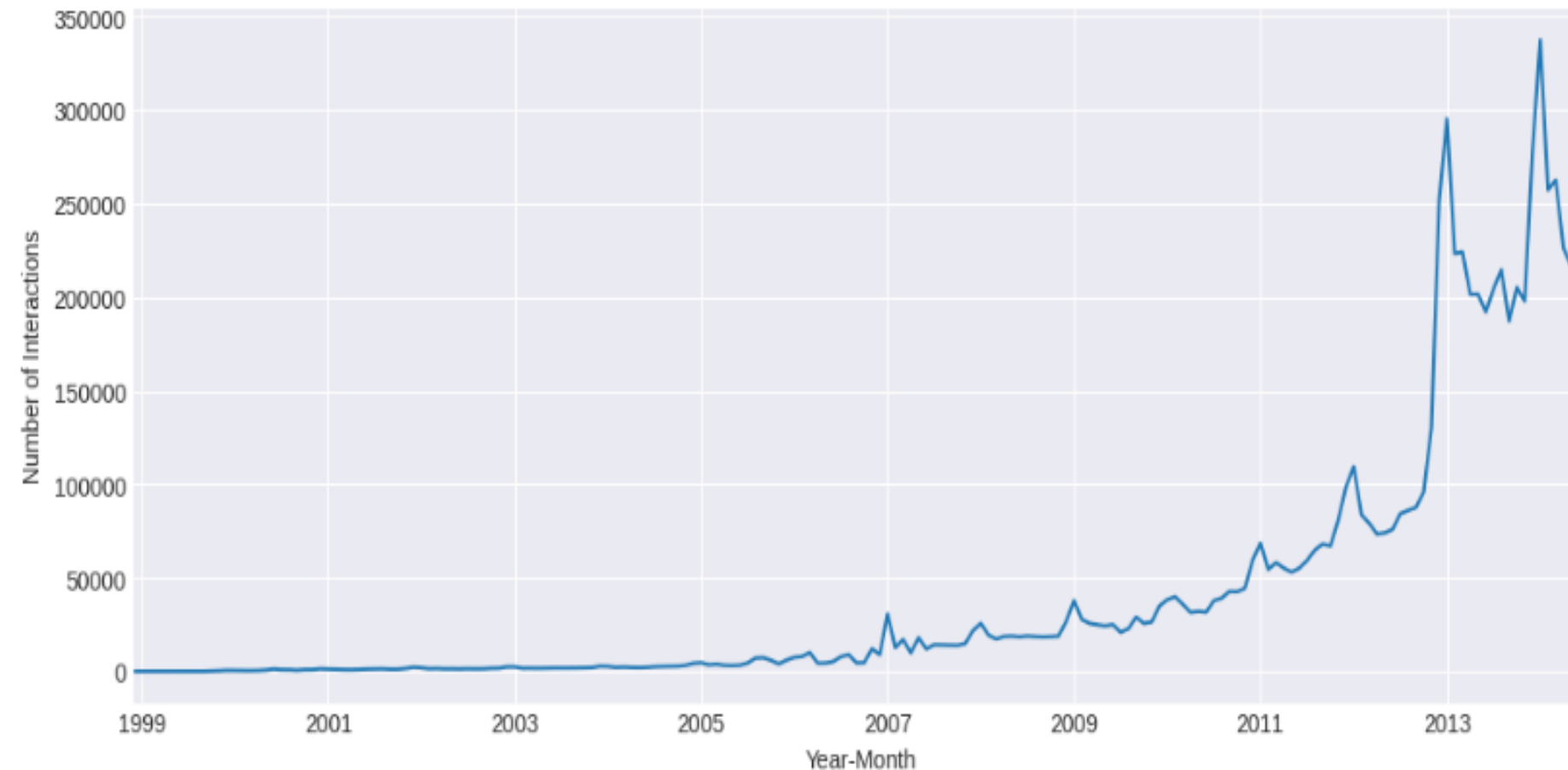


〈활성 사용자 상위 20명〉



시간별 판매 분석

〈월별 상호 평가〉



- 초기 기간 (1999년 ~ 2005년):
상호작용 수가 매우 낮고 비교적 안정적인
평탄한 모습

- 성장 초기 (2005년 ~ 2010년):
상호작용 수가 서서히 증가하기 시작하지만,
증가세는 완만

- 급격한 성장 (2010년 이후):
상호작용 수가 폭발적으로 증가
이는 데이터 수집 기간 중 가장 많은 활동이
일어났음을 의미

- 최신 추세 (2013년 후반):
최고점을 찍은 후 약간의 하락세를 보이지만,
여전히 이전 기간에 비해 매우 높은 수준을 유지

→ 이 서비스 또는 플랫폼은 2010년 이후에
사용자 활동이 폭발적으로 증가.
데이터 수집이 종료된 시점(약 2013년 중반)까지도
활발한 상호작용이 이어지고 있음.

Chapter 2. 데이터 분석

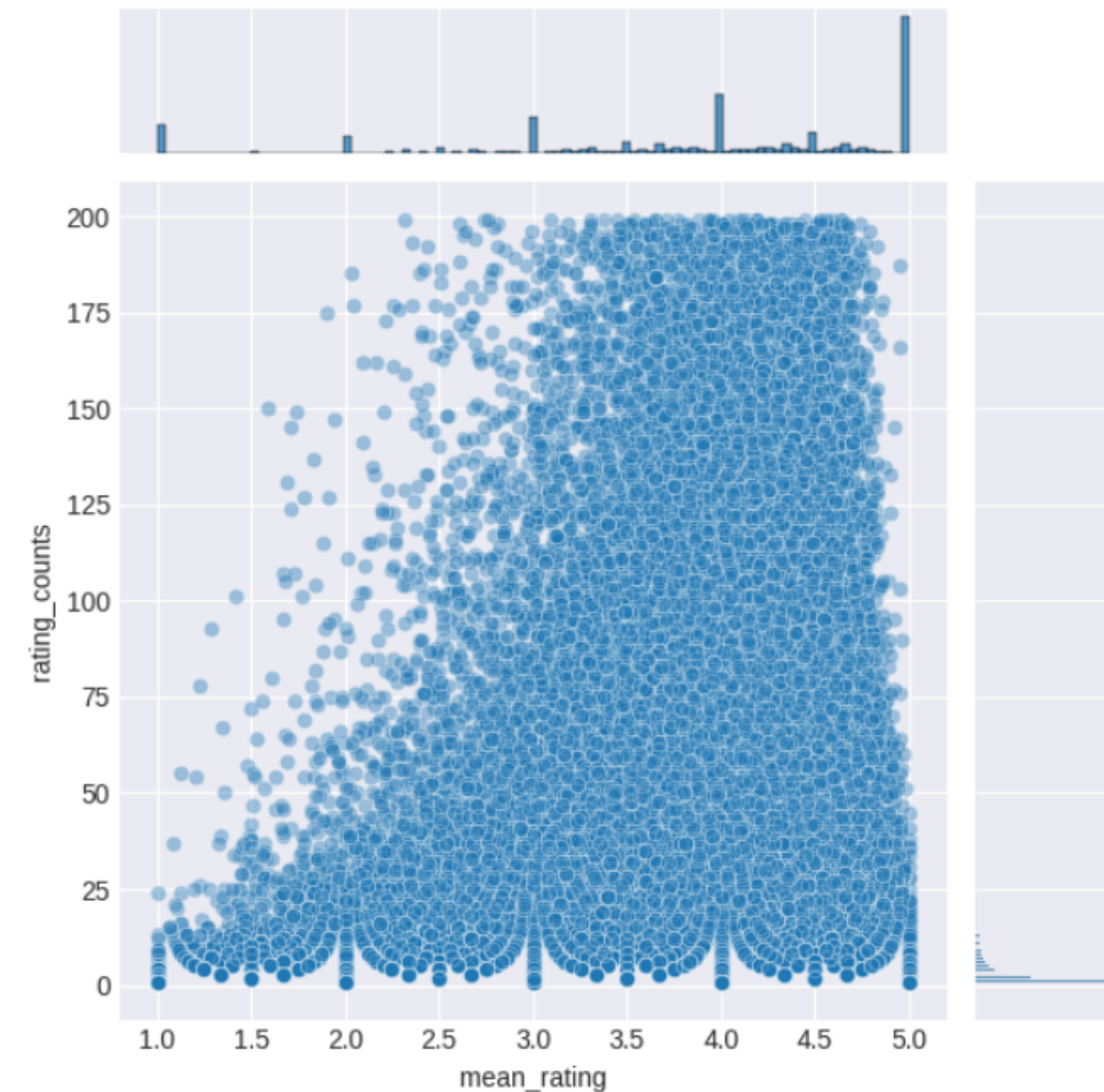
2

아이템별 평점 통계 계산 (상위 5개)

	rating_counts	mean_rating
productId		
B0074BW614	18244	4.491504
B00DROPDNE	16454	3.931020
B007WTAJTO	14172	4.424005
B0019EHU8G	12285	4.754497
B006GW05WK	12226	4.314657

평균 평점 VS 평점 수 Jointplot (평점 수가 너무 많은 outlier를 제외하고 분포를 봄)

```
print("\n--- 평균 평점 vs 평점 수 Jointplot ---")
plt.figure(figsize=(8, 6))
plt.rcParams['patch.force_edgecolor'] = True
# 평점 수가 200개 미만인 아이템들만으로 분포를 확인 (너무 큰 값 제외)
sns.jointplot(
    x='mean_rating',
    y='rating_counts',
    data=ratings_stats[ratings_stats['rating_counts'] < 200],
    alpha=0.4,
    kind='scatter' # 'hex' (밀도) 또는 'kde' (등고선)도 가능
)
plt.suptitle("평균 평점과 평점 수의 관계 (평점 200개 미만 아이템)", y=1.02)
plt.show()
```

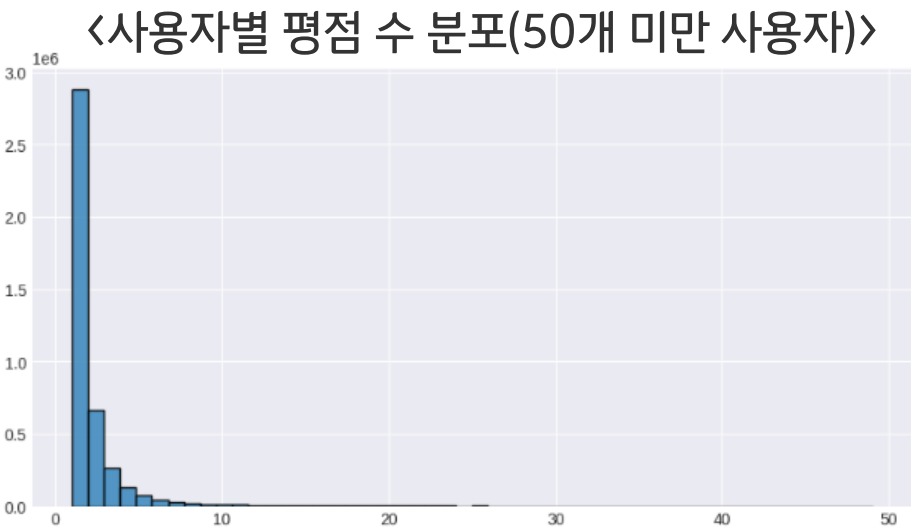


- X축: 평균 평점이 3.0에서 5.0 사이에 가장 많은 아이템이 분포
- Y축: 평점 수가 0에 가까운 부분에 압도적으로 많음
(대부분의 아이템이 200개 미만의 평점 중에서도 아주 적은 평점만을 받았음)

사용자별 평점 수 분포 (Long Tail)

- 대부분의 사용자가 적은 수의 평점을 남김

```
— 사용자별 평점 수 통계 (상위 5명) —
userId
A5JLAU2ARJ0B0    520
ADLVFFE4VBT8      501
A30XHLG6DIBRW8    498
A6FIAB28IS79      431
A680RUE1FD08B     406
Name: count, dtype: int64
```

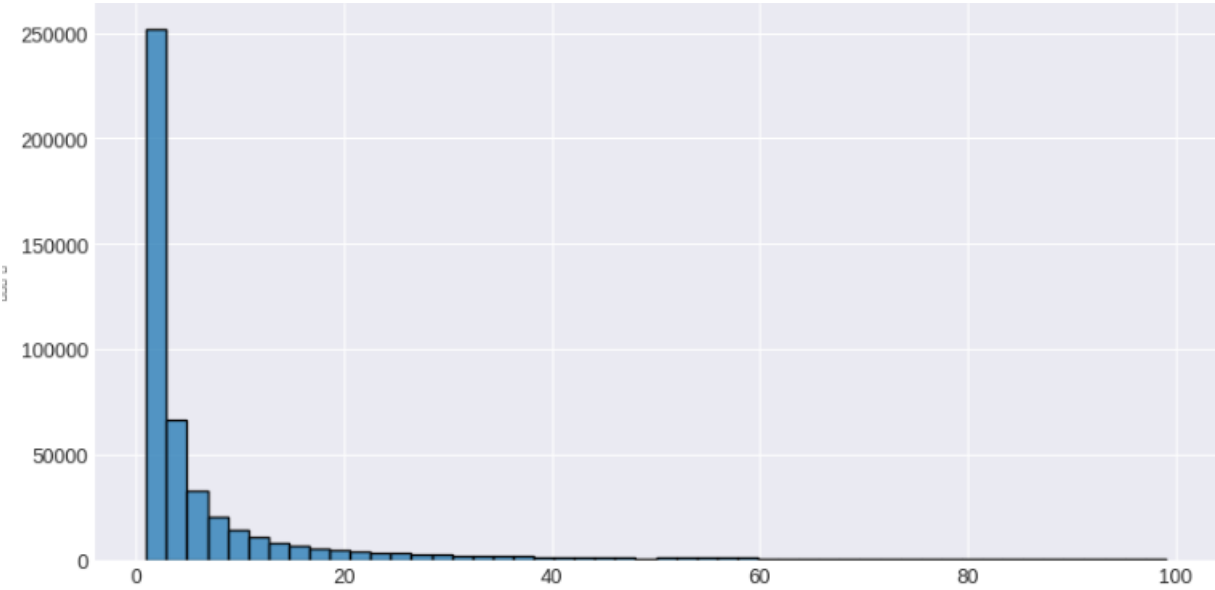


- 그래프의 막대 X축 값 0에 가까운 곳에 압도적으로 높게 솟아 있음.
- Y축의 사용자 수가 300만 명에 가까운 거대한 수치를 나타냄.
- 대부분의 사용자가 단 1개의 평점 또는 극히 적은 수 (1~5개 미만)의 평점만을 남겼음을 의미

아이템별 평점 수 분포 (Popularity)

- 소수의 아이템이 대부분의 평점을 받음

〈아이템별 평점 수 분포(100개 미만 아이템)〉



- 헤드 (Head): 소수의 매우 인기 있는 아이템 (평점 수 100개 이상 또는 수천·수만 개)이 존재하며 이들이 전체 평점의 대부분
- 긴 꼬리 (Long Tail): 대다수의 아이템(25만 개 이상)은 평점 수가 100개 미만, 대부분은 거의 주목받지 못함.
- 데이터의 희소성 문제 야기

Chapter 2. 데이터 분석

2

누적 사용자 비율 그래프(파레토 원리)

```
# 1. 원본 df에서 사용자별 평점 수(구매 수) 계산
user_interaction_counts = df['userId'].value_counts().sort_values(ascending=False)

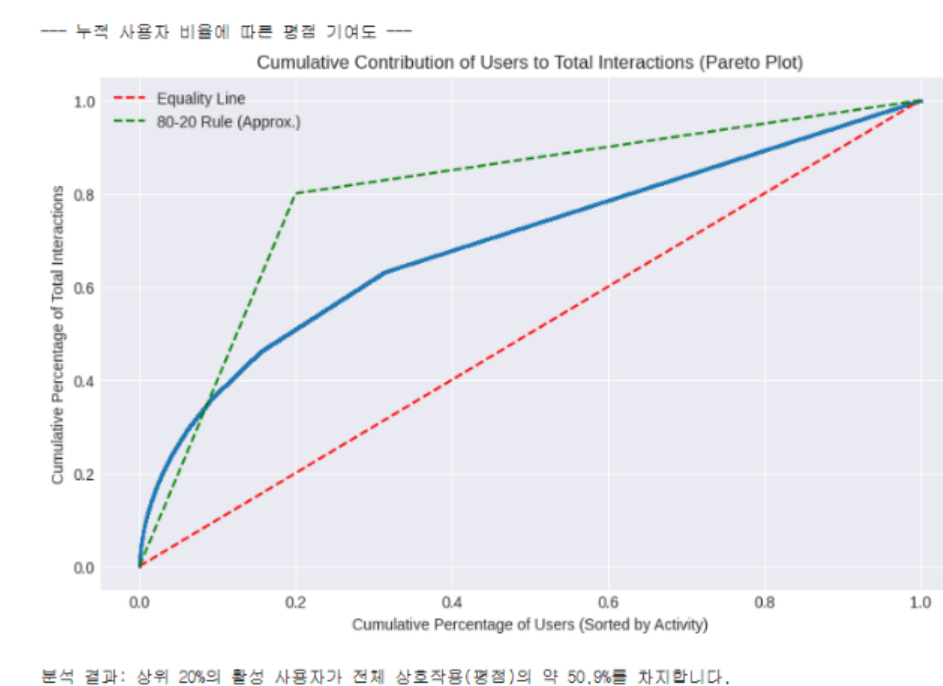
# 2. 누적 합계(Cumulative Sum) 계산
total_interactions = user_interaction_counts.sum()
cumulative_interactions = user_interaction_counts.cumsum()

# 3. 누적 비율 계산
total_users = len(user_interaction_counts)
# 1/N, 2/N, 3/N... (사용자 비율)
cumulative_user_ratio = np.arange(1, total_users + 1) / total_users
# (누적 평점 / 전체 평점) (평점 기여도 비율)
cumulative_interaction_ratio = cumulative_interactions / total_interactions

# 4. 시각화
plt.figure(figsize=(10, 6))
plt.plot(cumulative_user_ratio, cumulative_interaction_ratio,
         marker='.', markersize=1)
plt.title('Cumulative Contribution of Users to Total Interactions (Pareto Plot)')
plt.xlabel('Cumulative Percentage of Users (Sorted by Activity)')
plt.ylabel('Cumulative Percentage of Total Interactions')
plt.grid(True)

# 80-20 법칙을 시각화하기 위한 기준선
plt.plot([0, 1], [0, 1], 'r--', label='Equality Line') # 완전 균등 선
plt.plot([0, 0.2, 1], [0, 0.8, 1], 'g--', label='80-20 Rule (Approx.)') # 80-20 법칙 선
plt.legend()
plt.show()

# 5. 해석 출력 (예시)
try:
    # 상위 20%의 사용자가 몇 %의 상호작용을 차지하는지 계산
    top_20_percent_users_index = int(total_users * 0.2)
    contribution_of_top_20 =
    cumulative_interaction_ratio.iloc[top_20_percent_users_index]
    print(f"분석 결과: 상위 20%의 활성 사용자가 전체 상호작용(평점)의 약
    {contribution_of_top_20:.1%}를 차지합니다.")
except Exception as e:
    print(f"분석 결과 출력 중 오류: {e}")
```



1. Equality Line (적색 점선):
완벽하게 활동이 균등한 상태.
 2. 80-20 Rule (녹색 점선):
파레토 원리의 근사치로, 상위 20%의 사용자가 전체 상호작용의 80%를 차지하는 지점을 나타내는 이상적인 기준선
 3. 데이터 곡선 (파란색 실선):
실제 사용자들의 활동 기여도.
곡선이 균등선(적색 선)보다 훨씬 위에, 그리고 가파르게 솟아 있음.
- 50.9% 기여: 전체 사용자의 소수(20%)가 전체 평점의 절반 이상(50.9%)을 남김.

Chapter 3

데이터 전처리

1

전처리 목적

2

전처리

3

훈련/테스트 데이터 분리

전처리 목적

1. 데이터 희소성(Sparsity) 문제 극복

- 활동성 낮은 사용자와 인기 없는 아이템 사전에 제거.
- 모델이 학습해야 하는 행렬의 크기 대폭 축소 후 밀도 높임.

2. 콜드 스타트 문제 완화 및 신뢰도 확보

- 콜드 스타트 문제:
정보 부족하여 새로운 사용자나 아이템 추천하기 어려운 상황
- 사용자 필터링:
최소 10개 이상의 평점 남긴 사용자만 남겨
의미 있는 패턴 학습하도록 유도
- 아이템 필터링:
최소 20개 이상의 평점을 받은 아이템만 남겨
인기도와 품질 어느정도 검증된 아이템을 대상으로 추천

3. 모델 학습 효율성 및 성능 최적화

- 데이터 필터링을 통해 학습 리소스를 효율적으로 사용하고
모델의 예측 성능을 높임
- 효율성:
원본 780만 건의 데이터 대신 6.8만 건의 고품질 데이터만
사용함으로써 학습 시간과 비용을 절감하고 빠른 실험이 가능.
- 성능:
필터링된 데이터→ 노이즈와 낮은 신뢰도의 정보 제거함.
따라서 모델 더 명확하게 사용자-아이템 간 잠재적인 관계
파악해 추천 정확도 높임.

전처리

최소 상호작용 필터링

- 데이터의 희소성(Sparsity) 문제를 완화,
신뢰도가 낮은 극소수 평점에 기반한 추천을 방지

```
MIN_USER_RATINGS = 10
MIN_ITEM_RATINGS = 20
def filter_min_interactions(df, min_user, min_item):
    print(f"\n--- 최소 상호작용 필터링 시작 (User >= {min_user},
        Item >= {min_item}) ---")
    df_filtered = df.copy()

    # (반복 필터링 로직은 동일)
    while True:
        prev_shape = df_filtered.shape[0]

        user_counts = df_filtered['userId'].value_counts()
        valid_users = user_counts[user_counts >= min_user].index
        df_filtered = df_filtered[df_filtered['userId'].isin(valid_users)]

        item_counts = df_filtered['productId'].value_counts()
        valid_items = item_counts[item_counts >= min_item].index
        df_filtered = df_filtered[df_filtered['productId'].isin(valid_items)]

        if df_filtered.shape[0] == prev_shape:
            break
```

```
if len(df_filtered) == 0:
    print("경고: 필터링 결과 0행입니다.
        기준을 더 낮추거나 (예: 2, 2) 실제 데이터를 사용하세요.")

print(f"필터링 완료. 원본: {len(df)} -> 필터링 후: {len(df_filtered)}")
return df_filtered

filtered_df = filter_min_interactions(df, MIN_USER_RATINGS, MIN_ITEM_RATINGS)
```

코드 결과

```
--- 최소 상호작용 필터링 시작 (User >= 10, Item >= 20) ---
필터링 완료. 원본: 7824482 -> 필터링 후: 66060
```

*결과:

사용자와 아이템의 활동 기준(10/20개)을 적용한 결과,
약 99%의 데이터가 제거됨.
→ 대부분의 평점 데이터가 소수의 활성 사용자나
인기 아이템에 집중되어 있음

사용자/아이템 ID 인덱싱

- 필터링을 통해 남아있는 최종 사용자 및 아이템에 대해 컴퓨터가 처리하기 쉬운 연속적인 정수 ID를 부여하는 단계

```
user_encoder = LabelEncoder()
item_encoder = LabelEncoder()

if len(filtered_df) > 0:
    filtered_df['user_idx'] =
        user_encoder.fit_transform(filtered_df['userId'])
    filtered_df['item_idx'] =
        item_encoder.fit_transform(filtered_df['productId'])

    n_users_final = filtered_df['user_idx'].nunique()
    n_items_final = filtered_df['item_idx'].nunique()

    print(f"\n--- ID 인덱싱 완료 ---")
    print(f"최종 사용자 수: {n_users_final}, 최종 아이템 수: {n_items_final}")
else:
    print("\n--- ID 인덱싱 건너뛴 (필터링된 데이터 없음) ---")
    n_users_final = 0
    n_items_final = 0
```

코드 결과

--- ID 인덱싱 완료 ---

최종 사용자 수: 3769, 최종 아이템 수: 1598

*결과:

- 최소 10개 평점을 남긴 사용자→ 3,769명
- 최소 20개 평점을 받은 아이템→ 1,598개로 줄음.
- 3,769명의 사용자가 1,598개의 아이템에 대해 남긴 68,080개의 평점 데이터 추천 모델 학습을 위한 최종 데이터 됨.

훈련/테스트 데이터 분리

- 시간 순서를 기준으로 분리
- 추천 시스템에서 모델의 미래 예측 능력 평가하기 위해 중요

```
# 1. timestamp 기준으로 정렬
df_sorted = filtered_df.sort_values(by='timestamp', ascending=True)

# 2. 80% 훈련, 20% 테스트로 분리
train_size = int(len(df_sorted) * 0.8)
train_df = df_sorted.iloc[:train_size]
test_df_raw = df_sorted.iloc[train_size:]

# 3. 테스트셋 콜드 스타트(Cold Start) 제거
# 테스트셋에만 존재하는 (훈련셋에 없는) 사용자나 아이템은 예측이 불가능하므로 제거합니다.
train_users = set(train_df['user_idx'].unique())
train_items = set(train_df['item_idx'].unique())

test_df = test_df_raw[
    (test_df_raw['user_idx'].isin(train_users)) &
    (test_df_raw['item_idx'].isin(train_items))
]

print(f"\n--- 시간 기반 데이터 분리 완료 ---")
print(f"훈련 데이터: {len(train_df)} 행")
print(f"테스트 데이터 (원본): {len(test_df_raw)} 행")
print(f"테스트 데이터 (콜드 스타트 제거 후): {len(test_df)} 행")

# 평가를 위한 ground truth 생성
test_user_item_map =
test_df.groupby('user_idx')['item_idx'].apply(list).to_dict()

# 모든 모델의 평가 결과를 저장할 딕셔너리
results = {}
```

코드 결과

```
--- 시간 기반 데이터 분리 완료 ---
훈련 데이터: 52848 행
테스트 데이터 (원본): 13212 행
테스트 데이터 (콜드 스타트 제거 후): 9113 행
```

- 콜드 스타트 제거 영향:
원본 테스트 데이터(13,212행) 중 4,099행(약 32%) 제거.
- 테스트 시점의 약 1/3은
훈련 데이터에 없던 새로운 사용자나 아이템에 의한 것

Chapter 4

모델링

1 Baseline

2 itemCF

3 SVD

4 ALS

5 GNN

모델 평가 지표

A 모델 예측의 정확도 측정 (RMSE/MAE)

모델이 사용자의 평점을 얼마나 정확하게 예측하는지 측정

RMSE	실제 평점과 예측 평점 간의 오차를 측정 큰 오차에 더 큰 가중치 부여
MAE	실제 평점과 예측 평점 간의 오차의 절댓값 측정 RMSE에 비해 이상치의 영향을 덜 받음

B Top-K 추천 목록의 품질 측정

추천 시스템의 '실제 유용성' 측정

Precision@K	추천 목록 K개 중 사용자가 실제로 선호하는 아이템의 비율
Recall@K	사용자가 실제로 선호하는 모든 아이템 중 추천 목록 K개에 포함된 아이템의 비율

모델 종류

모델	특징
Baseline	전체 평균 평점에 사용자별 편향과 아이템별 편향을 더해 평점을 예측하는 가장 기본적인 통계 모델
ItemCF	사용자가 과거에 선호했던 아이템과 유사한 아이템을 찾아 추천하는 이웃 기반 모델
SVD	사용자와 아이템을 소수의 잠재 요인 벡터로 분해하고, 이 잠재요인들의 내적으로 평점을 예측하는 행렬분해 모델
ALS	사용자 잠재 요인과 아이템 잠재 요인을 교대로 번갈아 가며 최소 제곱 오차를 줄이는 방식으로 학습하는 모델
GNN	사용자와 아이템을 노드로 연결한 상호작용 그래프를 구축하고, 그래프 신경망을 통해 이웃 노드의 정보를 통합하여 임베딩을 학습하는 모델

Baseline

접근방식

전체 평균 평점에 사용자 및 아이템의 편향을 더해 평점을 예측하는 방식으로 구현
확률적 경사 하강법을 이용해 학습

모델 파라미터

전체 평균 평점 / 사용자 편향 벡터 / 아이템 편향 벡터

[SGD 기반 편향 최적화 로직]

```
for epoch in range(n_epochs):
    for row in df.itertuples():
        u, i, r = row.user_idx, row.item_idx, row.rating
        pred = self.mu + self.bu[u] + self.bi[i]
        err = r - pred

        #확률적 경사 하강법
        self.bu[u] += lr * (err - self.reg * self.bu[u])
        self.bi[i] += lr * (err - self.reg * self.bi[i])
```

[Baseline 모델의 최종 예측 로직]

```
def predict(self, u, i):
    pred = self.mu
    if u < self.n_users: pred += self.bu[u]
    if i < self.n_items: pred += self.bi[i]
    return np.clip(pred, 1.0, 5.0) #평점 범위 제한
```


Baseline

하이퍼 파라미터

정규화 계수=0.02 학습률=0.01 에포크 횟수=10

[모델 생성 단계-정규화 계수 설정]

```
def __init__(self, reg=0.02):  
    ~
```

[모델학습-학습률, 에포크 횟수 설정]

```
def fit(self, df, n_users, n_items, n_epochs=10, lr=0.01):  
    ~
```

성능 평가

RMSE	MAE	Precision@10	Recall@10
0.8641	0.6118	0.0028	0.0059

FunkSVD

접근방식

FunkSVD라는 행렬 분해 기법을 확률적 경사 하강법을 기반으로 구현
Baseline 모델의 편향 보정을 그대로 활용하면서,
사용자-아이템 간의 숨겨진 협업적 관계를 파악하는 **잠재요인** 추가

모델 파라미터

전체 평균 평점 / 사용자 편향 벡터 / 아이템 편향 벡터
사용자의 잠재요인 벡터 / 아이템의 잠재요인 벡터

[잠재요인 SGD 업데이트 핵심 로직]

```
pu = self.P[u].copy()
qi = self.Q[i].copy()
self.P[u] += self.lr * (err * qi - self.reg * pu)
self.Q[i] += self.lr * (err * pu - self.reg * qi)
```

[FunkSVD의 최종 예측 공식 및 로직]

```
def predict(self, u, i):
    if u >= self.n_users or i >= self.n_items:
        return self.baseline.mu

    # 최종 예측 = 전역 평균 + 사용자 편향 + 아이템 편향 + 잠재요인 내적
    pred = self.baseline.mu + self.baseline.bu[u] + self.baseline.bi[i] + self.P[u].dot(self.Q[i])
    return np.clip(pred, 1.0, 5.0)
```

FunkSVD

하이퍼 파라미터

정규화 계수=0.02 학습률=0.07 에포크 횟수=20 잠재 요인 개수=50

[초기화 단계-하이퍼 파라미터 설정]

```
def __init__(self, n_factors=50, lr=0.007, reg=0.02, n_epochs=20):
    ~
```

성능 평가

RMSE	MAE	Precision@10	Recall@10
0.8871	0.6109	0.0024	0.0057

ItemCF

접근방식

특정 아이템과 유사한 다른 아이템들에 대한 사용자의 평가를 가중 평균하여 아이템에 대한 사용자의 평점을 예측
예측의 안정성을 높이기 위해 Baseline모델을 내장하여 결측치나 계산 실패시의 대안 예측을 강화함

모델 파라미터

전체 평균 평점 / 사용자 편향 벡터 / 아이템 편향 벡터
아이템-사용자 행렬 / 사용자별 평균 평점

[Baseline 모델 내장 학습]

```
self.baseline.fit(df, n_users, n_items, n_epochs=10, lr=0.01)
```

[희소 행렬 생성]

```
self.item_user_matrix =  
csr_matrix((df['rating'].values,  
(df['item_idx'].values, df['user_idx'].values)),  
           shape=(n_items, n_users))
```

[코사인 유사도 계산]

```
sim_scores = cosine_similarity(target_item_vec, rated_items_vecs).ravel()
```

[ItemCF의 예측 평점 최종 로직]

```
for idx in top_k_indices:  
    similarity = sim_scores[idx]  
    rating = user_ratings[idx]  
  
    if similarity > 0:  
        numerator += similarity * rating  
        denominator += similarity
```

ItemCF

하이퍼 파라미터

*baseline 정규화 계수=0.02 학습률=0.01 에포크 횟수=10
*ItemCF 유사 이웃 개수=40

[baseline- 정규화 계수 설정]

```
self.baseline = BaselinePredictor(reg=0.02)
```

[baseline- 학습률, 에포크 횟수 설정]

```
self.baseline.fit(df, n_users, n_items, n_epochs=10, lr=0.01)
```

[ItemCF- 유사 이웃 개수 설정]

```
def __init__(self, k_sim=40):  
    ~
```

성능 평가

RMSE	MAE	Precision@10	Recall@10
1.0174	0.6435	nan	nan

ALS-implicit 기반

접근방식

행렬 분해를 ALS 알고리즘을 사용하여 수행
전체 평점 행렬에 ALS를 적용하는 대신, Baseline 모델이 예측하지 못한 오차에 대해서만 잠재요인을 학습하는 잔차학습 이용
사용자 또는 아이템을 번갈아가며 행렬 연산을 통해 한 번에 최적의 해를 찾아냄

모델 파라미터

사용자 잠재 요인 행렬 (ALS) / 아이템 잠재 요인 행렬
(ALS) 편향 파라미터 (Baseline)

[잔차 학습]

```
residuals = []
for row in df.itertuples():
    u, i, r = row.user_idx, row.item_idx, row.rating
    bias_pred = self.baseline.mu + self.baseline.bu[u] + self.baseline.bi[i]
    residuals.append(r - bias_pred)
```

[잔차를 이용해 희소행렬 생성]

```
item_user_residual_sparse = csr_matrix(
    (df_residual['rating'].astype(np.float32),
     (df_residual['item_idx'].values, df_residual['user_idx'].values)),
    shape=(n_items, n_users))
```

[최종 예측 로직 (하이브리드형)]

```
# 편향예측-baseline 이용
bias_pred = self.baseline.mu + self.baseline.bu[u] + self.baseline.bi[i]
# 잔차예측-ALS 이용
item_vec = self.model.user_factors[i]
user_vec = self.model.item_factors[u]
residual_pred = user_vec.dot(item_vec)
# 최종예측=편향+잔차
prediction = bias_pred + residual_pred
return np.clip(prediction, 1.0, 5.0)
```

ALS-implicit 기반

하이퍼 파라미터

*Baseline 정규화 계수=0.02 학습률=0.01 에포크 횟수=10
*ALS 알고리즘 반복 횟수=15 정규화 계수=0.01 잠재요인 개수=50

[baseline- 정규화 계수 설정]

```
self.baseline = BaselinePredictor(reg=0.02)
```

[baseline- 학습률, 에포크 횟수 설정]

```
self.baseline.fit(df, n_users, n_items, n_epochs=10, lr=0.01)
```

[ALS- 하이퍼 파라미터 설정]

```
def __init__(self, factors=50, regularization=0.01, iterations=15)
```

성능 평가

RMSE	MAE	Precision@10	Recall@10
0.8641	0.6109	0.0029	0.0062

GNN

접근방식

추천 문제를 그래프에서의 링크 예측 문제로 해결하며, 기존의 행렬분해모델보다 더 풍부한 이웃정보를 활용
사용자 및 아이템: 노드, 상호작용: 엣지(연결)로 표현
GCN레이어를 사용하여 각 노드의 초기 임베딩에 연결된 이웃노드의 정보를 반복적으로 집계->통합
사용자 및 아이템의 최종 임베딩에 그래프 구조 정보 즉, 협업적 관계가 깊이 반영되도록

모델 파라미터

노드 임베딩 행렬 / GCN 레이어 가중치 / GCN 레이어 편향

[GCN 레이어 정의, 노드 임베딩 초기화]

```
class GCNRec(Module):
    def __init__(self, num_nodes, embedding_dim, hidden_dim):
        super().__init__()
        # 모든 노드 (사용자+아이템)의 초기 임베딩 테이블
        self.embedding = Embedding(num_nodes, embedding_dim)
        # 메시지 전달을 수행하는 GCN 레이어
        self.conv1 = GCNConv(embedding_dim, hidden_dim)
        self.conv2 = GCNConv(hidden_dim, hidden_dim)
```

[임베딩 학습: 그래프 인코딩]

```
def encode(self, edge_index):
    x = self.embedding.weight # 노드 임베딩 벡터
    x = self.conv1(x, edge_index).relu() # 1단계 GCN&ReLU 활성화
    z = self.conv2(x, edge_index) # 2단계 GCN (최종 임베딩)
    return z
```


GNN

[디코딩-상호작용 점수 예측]

```
def decode(self, z, edge_label_index):
    user_emb = z[edge_label_index[0]] #사용자 잠재 요인
    item_emb = z[edge_label_index[1]] #아이템 잠재 요인
    return (user_emb * item_emb).sum(dim=-1) #내적 연산 (유사도 측정)
```

[학습 최적화]

```
#긍정 엣지와 부정 엣지결합
edge_label_index = torch.cat([pos_edge_index, neg_edge_index], dim=1)
edge_label = torch.cat([
    torch.ones(pos_edge_index.size(1)), #긍정 상호작용 레이블 (1)
    torch.zeros(neg_edge_index.size(1)) #부정 상호작용 레이블 (0)
], dim=0)

out = model.decode(z, edge_label_index)

#이진 분류를 위한 손실함수
loss = F.binary_cross_entropy_with_logits(out, edge_label)

loss.backward() #역전파
optimizer.step() #파라미터 업데이트
```

GNN

하이퍼 파라미터

임베딩 차원=64 은닉층 차원=64 학습률=0.01 에포크 횟수=30

[GNN- 하이퍼 파라미터 설정]

```
EMBEDDING_DIM = 64
HIDDEN_DIM = 64
GNN_EPOCHS = 30

gnn_model = GCNRec(num_nodes, EMBEDDING_DIM, HIDDEN_DIM).to(DEVICE)
optimizer = torch.optim.Adam(gnn_model.parameters(), lr=0.01)
```

성능 평가

RMSE	MAE	Precision@10	Recall@10
N/A	N/A	0.0004	0.001

Chapter 5

하이퍼파라미터 튜닝

1

ALS 모델에서의 하이퍼파라미터 튜닝

ALS 모델에서의 하이퍼파라미터 튜닝

*내장된 baseline은 이미 최적의 하이퍼파라미터로 학습되었다고 가정

하이퍼파라미터	의미
잠재 요인 개수	모델의 표현능력을 결정하는 가장 중요한 요소 최적의 복잡도를 찾는다 - 너무 작으면 과소적합, 너무 크면 과적합 위험 존재
정규화 계수	모델의 과적합을 방지하는 역할 손실 함수에 추가되어 파라미터의 크기를 제한함
알고리즘 반복 횟수	ALS 알고리즘이 사용자 요인 행렬과 아이템 요인 행렬을 교대로 학습하는 전체 횟수 모델이 충분히 수렴할 수 있는 최소 횟수를 찾음

ALS 모델에서의 하이퍼파라미터 튜닝

〈하이퍼파라미터 후보군〉

잠재 요인 개수	정규화 계수	RMSE
30	0.01	0.8639
30	1.0	0.8639
30	10.0	0.8641
50	0.01	0.8641
50	1.0	0.8640
50	10.0	0.8641

Chapter 6

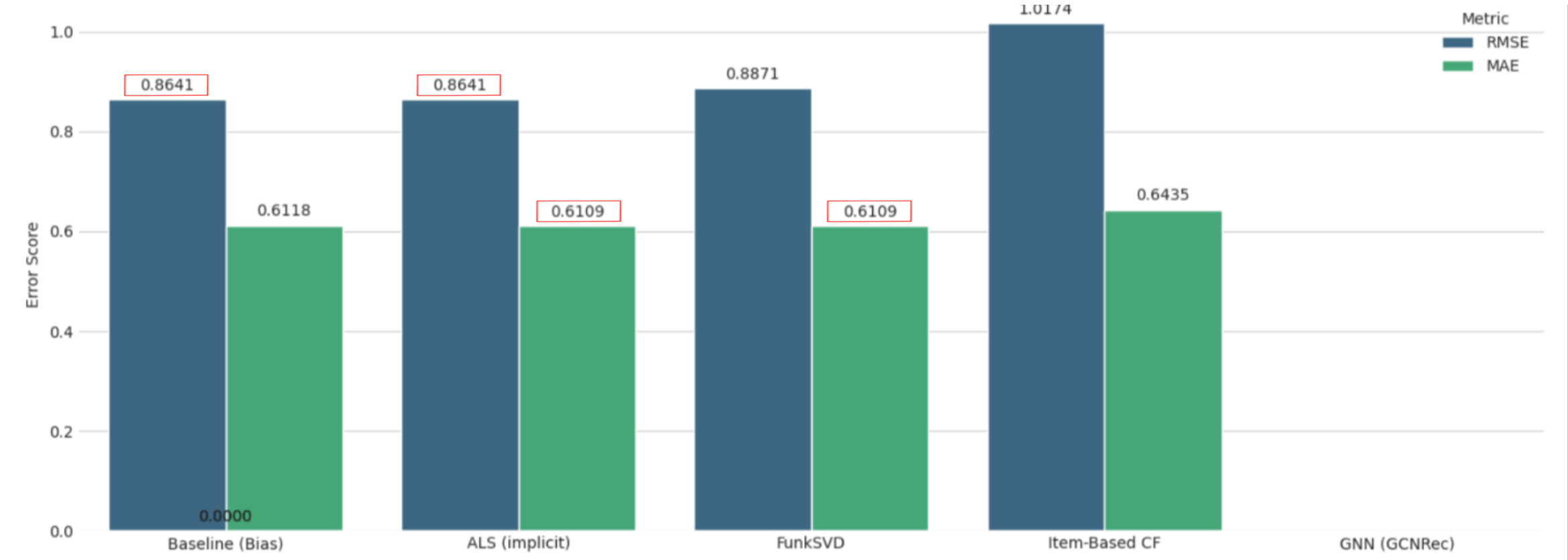
최종 결과 비교 및 결론

1

최종 결론

최종 결론

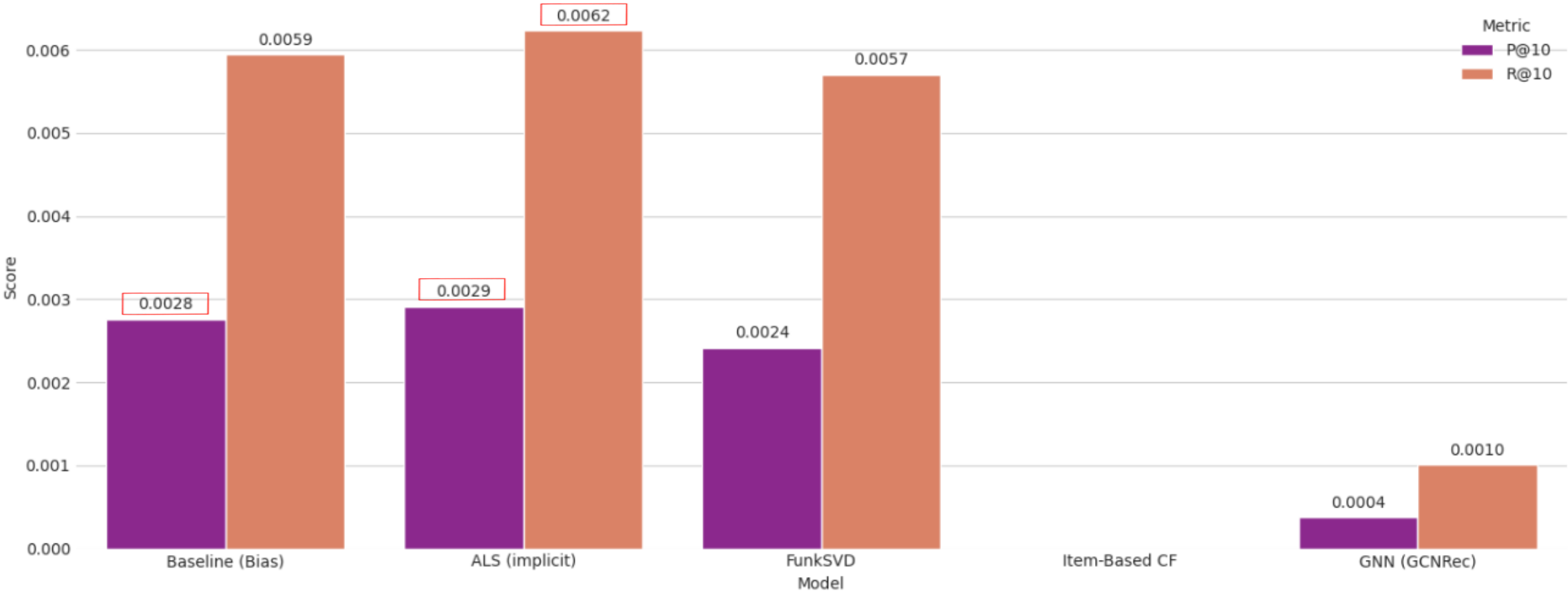
〈각 모델의 RMSE/MAE〉



성능지표	가장 좋은 모델
RMSE	Baseline과 ALS 모델이 RMSE 0.8641로 가장 좋음
MAE	ALS와 FunkSVD 모델이 MAE 0.6109로 가장 좋음 하지만, Baseline 0.6118과 매우 근소한 차이

최종 결론

〈각 모델의 P@10/R@10〉



성능지표	가장 좋은 모델
Precision@10	ALS 모델이 P@10 0.0029로 가장 좋음 하지만 Baseline (0.0028)과 매우 근소한 차이
Recall@10	ALS 모델이 R@10 0.0062로 가장 좋음

가장 단순한 모델이 가장 효율적

기술적으로는 **ALS 모델**이 RMSE와 P@10 모두에서 가장 균형 잡힌 최상위 성능을 보여줌

하지만 **Baseline 모델**은 단순히 사용자 고유 편향 (평점을 짜게/후하게 주는지)과 아이템고유 편향 (인기/비인기)만 계산한 모델임에도 불구하고, 복잡한 행렬분해 (ALS)모델과 거의 동일한 RMSE 및 P@10성능을 달성

따라서, 비용대비 성능을 고려할때, **Baseline 모델**이 가장 합리적인 선택
약간의 성능 향상이라도 중요한 상황이라면 모든 성능지표에서 근소하게 우위인 **ALS 모델**을 최종모델로 선택가능



감사합니다

KUGGLE 2차 프로젝트 - 문형주 강지윤 이서경 이현정